LOW LATENCY AND EFFICIENT VOICE ACTIVATOR FOR EDGE DEVICES

BY

2411EC010001	A.AKILA
2411EC010006	E.NAGESH
2411EC010013	D.DEVI SRI GANGA
2411EC010028	K.SNEHAL
2411EC010022	G.ARVIND
2411EC010040	P.SAIVARDHAN
2411EC010056	SYED AKIF

Under the Guidance of

Department of ECE
School of Engineering
Malla Reddy University
Hyderabad, Telangana, INDIA

ABSTRACT

The**Low-Latency andEfficientVoice Activator for Edge Devices** is a voice-based activation system designed to detect a predefined wake word or voice command locally on an edge device. Unlike conventional voice assistants that depend heavily on cloud servers, the proposed system performs the initial voice detection and processing directly on the device. This reduces response latency, minimizes network dependency, and improves privacy.

The system continuously monitors audio input through a microphone using a lightweight voice-processing algorithm. When the specified activation word is detected, the system triggers the required action or passes the command to the next processing stage. The system is designed to consume low computational power and memory, making it suitable for resource-constrained devices such as microcontrollers, embedded systems, smart appliances, and IoT devices.

1. Introduction

1.1 Problem Definition & Description

- Modern voice assistants generallydepend on cloud-based speech-processing services. This approach can introduce several problems, including **high latency, internet dependency, increased bandwidth usage, privacy concerns, and higher power consumption**.
- For edge devices with limited processing capabilities, continuously sending audio data to a remote server is inefficient. There is therefore a need for a lightweight voice activation system capable of detecting a predefined wake word locally and quickly.
- The proposed project addresses this problem by developing a **low-latency, computationally efficient voice activator** that performs wake-word detection directly on an edge device. Only after activation is detected does the system initiate further processing or communication.

1. Introduction

1.2 Objectives of the Project

- The major objectivesare:
- To develop a **real-time voice activation system** for edge devices.
- To detect a predefined **wake word/activation phrase** accurately.
- To minimize the **response latency** between the spoken command and activation.
- To reduce dependency on **internet connectivity and cloud processing**.
- To minimize **CPU, memory, and power consumption**.
- To perform voice processing locally for improved **privacy and security**.
- To design a system that can operate on **resource-constrained embedded hardware**.
- To reduce false activations caused by background noise and unintended speech.
- To provide a scalable architecture that can be integrated with **IoT devices, robots, smart appliances, and embedded systems**.

1. Introduction

1.3 Aim of the Project

- Themainaim of this project is to **design and implement a low-latency, energy-efficient, and privacy-friendly voice activation system for edge devices that can detect a predefined wake word locally without continuously depending on cloud services.**

2. Methodology

Step 1: Audio Acquisition

Amicrophone continuously captures the surrounding audio signal.

Step 2: Preprocessing

The captured audio is converted into a suitable digital format. Noise reduction, normalization, filtering, and audio framing can be applied to improve signal quality.

Step 3: Feature Extraction

Useful characteristics of the audio signal are extracted. Depending on the implementation, features such as **MFCCs, Mel-spectrograms, or other lightweight audio features** can be used.

Step 4: Wake-Word Detection

The extracted features are supplied to a lightweight voice/wake-word detection model. The model determines whether the predefined activation word has been spoken.

Step 5: Decision Making

The system compares the detection result with a predefined confidence threshold.

Wake word detected → Activate system

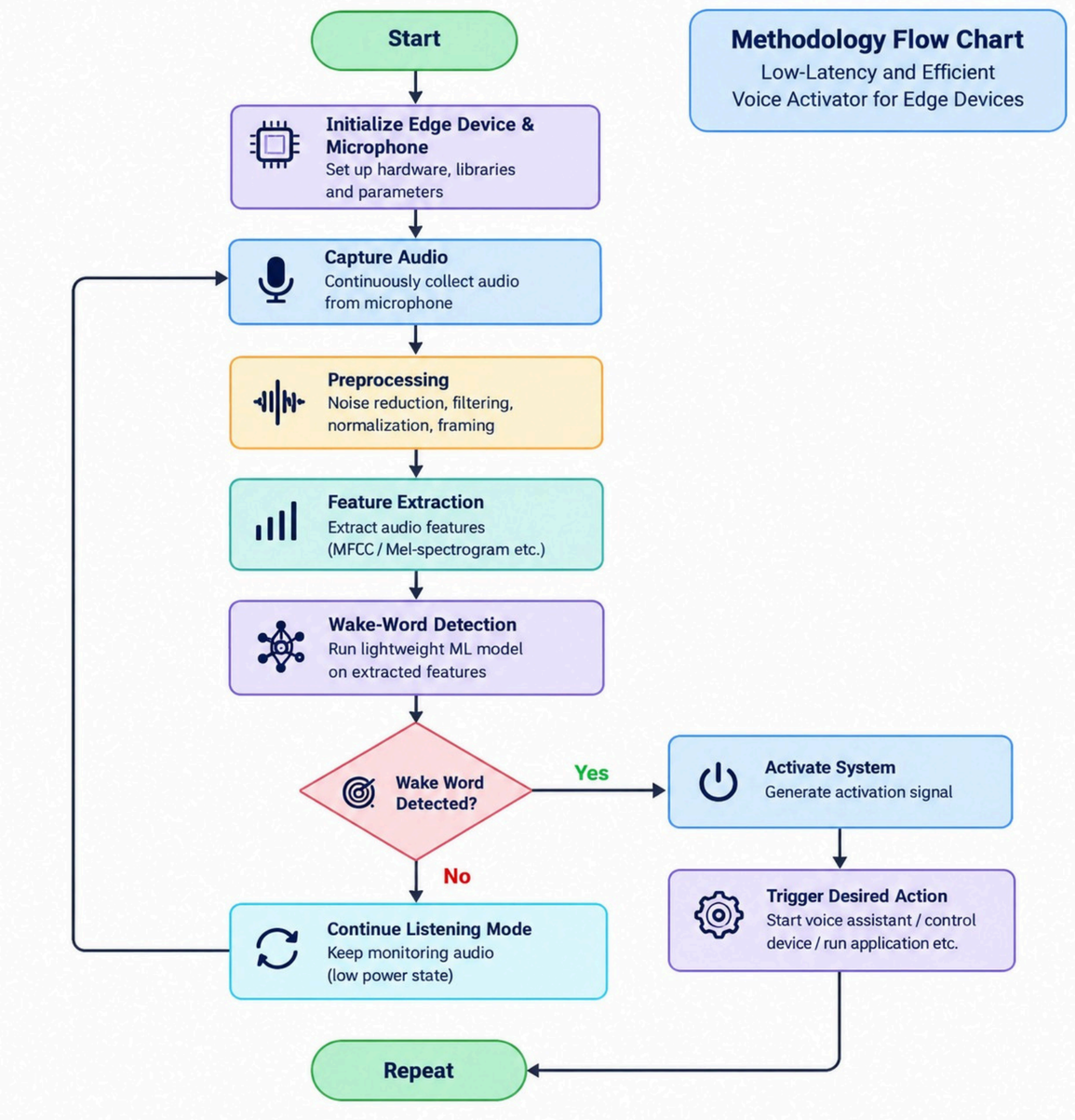
Wake word not detected → Continue monitoring

Step 6: Trigger Action

When the wake word is detected, the edge device generates an activation signal. This can be used to start speech recognition, control a robot, operate an appliance, or perform another predefined function.

Step 7: Return to Listening Mode

2.1 Methodology



3. Requirements

3.1 Hardware Requirements

- Processor : Intel i5
- Ram : 8GB
- Hard Disk : 150 GB

3. Requirements

3.2 Software Requirements

- Front End : HTML , CSS, Java Script
- Backend : Python
- Database : SQL(Structured Query language)
- Operating System : Windows 11

4. System Analysis

Thesystemcan beanalyzed basedon **latency, accuracy, computational resources, power consumption, connectivity, and privacy**.

Input
The microphone receives:
Human speech
Wake words
Background noise
Environmental sounds

Processing
The edge device performs:
Audio sampling
Preprocessing
Feature extraction
Wake-word classification
Confidence evaluation

4. System Analysis

Output

When the wake word is detected, the system produces an activation signal that can:

- **Start a voice assistant**
- **Activate another application**
- **Control an IoT device**
- **Start speech recognition**
- **Trigger a robot**
- **Control a smart appliance**

5. System Analysis

5.2 Existing System

Limitations Of Existing System

- **Limited processing capability**–Edge devices have fewer computational resources than computers or cloud servers.
- **Detection accuracy can decrease in high noise** – Very noisy environments may affect wake-word detection.
- **Limited vocabulary** – A lightweight activator is generally designed for a small set of predefined activation words.
- **Model size constraints** – The ML model must be small enough to fit within the device's memory and processing limits.
- **Hardware dependency** – Microphone quality and edge-device capabilities affect performance.
- **Language limitations** – The system may initially support only selected languages or accents.
- **False activations** – Similar-sounding words or environmental sounds can occasionally trigger the system.
- **Training requirement** – A suitable dataset may be required to achieve reliable wake-word detection.
- **Power constraints** – Battery-powered devices require careful optimization of continuous microphone operation.

5. System Analysis

- **Mobile App Platform:** A dedicated mobile app for easy and convenient access anytime, anywhere.
- **Real-Time Updates:** Provides accurate information on room availability and pricing instantly.
- **Advanced Filters:** Allows users to search hostels by budget, location, facilities, and other preferences.
- **Gender Preference Options:** Offers filtering for male-only, female-only, or co-ed hostels.
- **User-Friendly Design:** Intuitive interface for a smooth and hassle-free experience.
- **Verified Information:** Ensures reliable and up-to-date data for making informed decisions.
- **Comparison Feature:** Helps users compare multiple hostels side by side for better decision-

5. System Analysis

5.3 Proposed System

Advantages of proposed system

- **Low latency** – Local processing eliminates much of the network communication delay.
- **Offline operation** – Wake-word detection can work without an internet connection.
- **Improved privacy** – Audio does not need to be continuously transmitted to a cloud server.
- **Low bandwidth usage** – Continuous audio streaming to the cloud is avoided.
- **Energy efficient** – The system can remain in a lightweight monitoring state.
- **Fast response** – Local inference enables rapid activation.
- **Suitable for IoT** – Can be integrated into smart devices and embedded systems.
- **Reduced cloud dependency** – Basic voice activation does not require an external server.
- **Scalable** – The same concept can be adapted to different edge platforms.
- **Cost effective** – Can be implemented using relatively inexpensive embedded hardware.

5. System Analysis

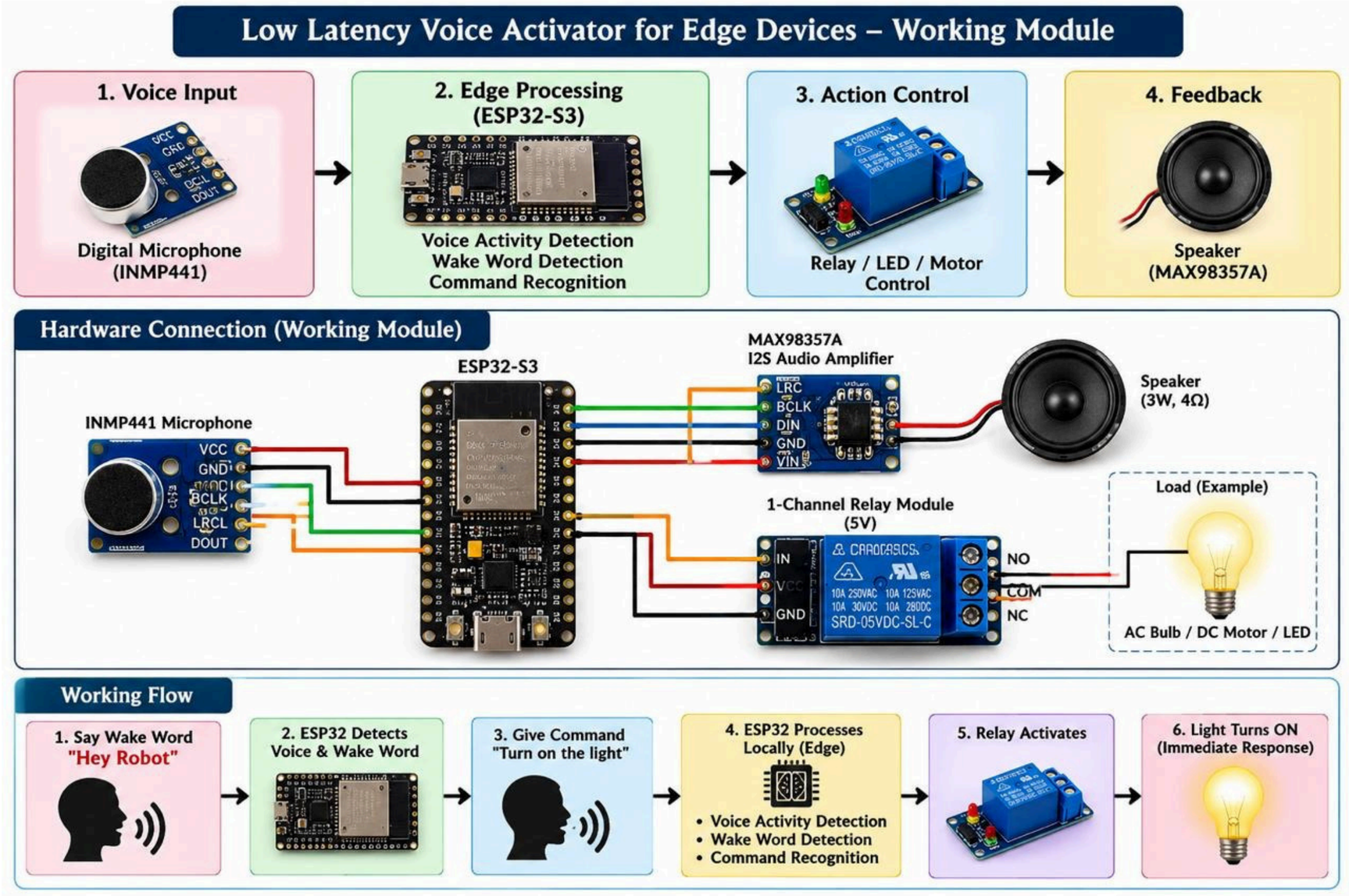
5.3 Proposed System

Key Features of proposed system

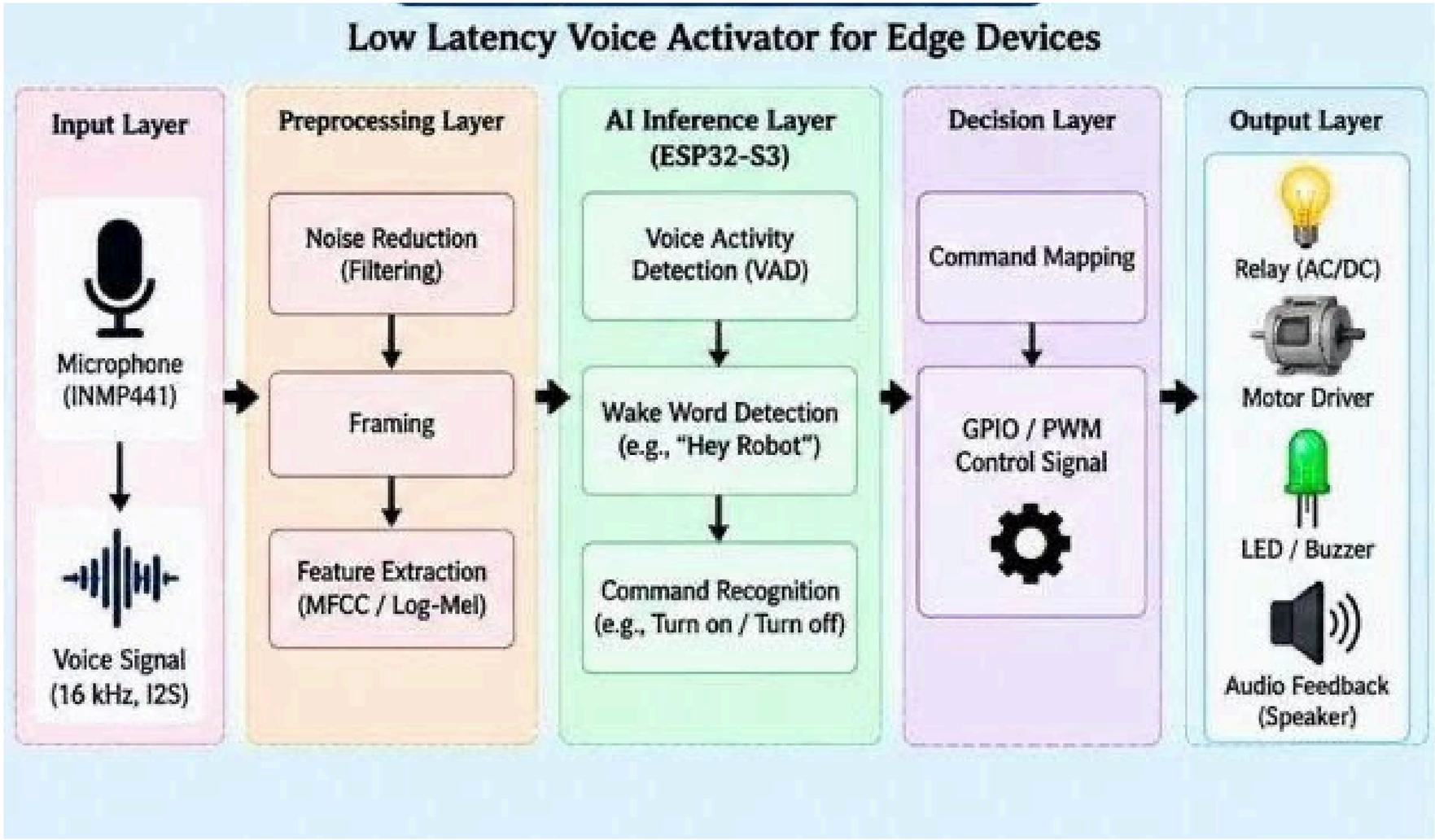
- Real-time wake-word detection
- Low-latency response
- Local edge processing
- Offline activation
- Low memory requirement
- Low computational overhead
- Low power consumption
- Noise-tolerant voice detection
- Configurable wake word
- Privacy-preserving architecture
- IoT and robotics integration
- Fast activation signal generation
- Lightweight machine-learning model
-

6.ARCHITECTURE

5



6.ARCHITECTURE



TESTING :

Hardware Testing :

- Verified ESP32-S3 power supply and GPIO connections.
- Tested INMP441 microphone for voice input.
- Checked OLED display output.
- Verified LED and speaker connections.

Voice Recognition Testing :

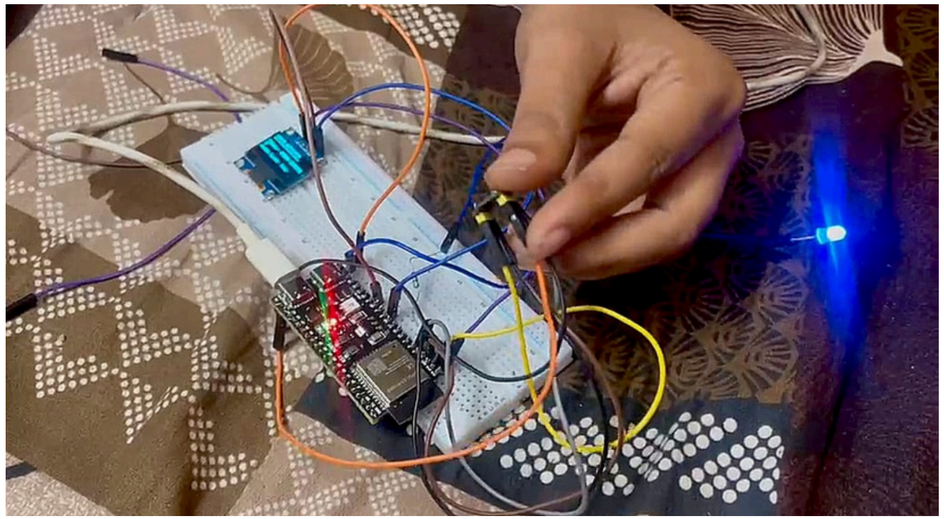
- Tested voice commands: LIGHT ON and LIGHT OFF.
- Checked microphone audio input and command detection.
- Verified response to different voice inputs.

Response Time Testing :

- Measured the time between giving a voice command and LED activation.
- Checked the system's response speed and reliability.

Functional Testing :

- Add a little bit of bodyVerified LED ON/OFF operations.
- Tested speaker voice feedback.
- Checked OLED status updates.
- Tested automatic LED switching after 30 seconds.



RESULT :

1. Voice Command Detection :

- Successfully detected LIGHT ON and LIGHT OFF commands during testing.
- Microphone captured voice input using INMP441.

2. LED Control :

- LED turned ON when the LIGHT ON command was detected.
- LED turned OFF when the LIGHT OFF command was detected.

3. OLED Display Output :

- Displayed the recognized command and current LED status.
- Provided real-time visual feedback.

4. Audio Feedback :

Speaker provided confirmation messages for ON and OFF operations.

5. Edge Processing :

- Voice processing was performed locally on the ESP32-S3.
- Reduced dependency on cloud-based processing and internet connectivity.

Thank you